

ResumelQ: AI-Based Resume Screening & Job Matching System

ABSTRACT

ResumelQ automates resume screening and job matching. It extracts skills, education and experience from PDF, DOC, DOCX and TXT resumes using NLP, converts text to TF-IDF features, and applies Naive Bayes, SVM and cosine similarity to rank resumes against a job description, delivering an explainable shortlist and role recommendations through a Flask web app.

PROBLEM STATEMENT

- Recruiters get **hundreds of resumes** per role; manual screening is slow and inconsistent.
- Human error and bias can **overlook qualified candidates** and delay hiring.
- Resumes come in **varied formats and layouts**, so unstructured text is hard to compare.
- Candidates rarely know **which roles fit** their skills.

OBJECTIVES

- Accept resumes in **PDF, DOC** (plus DOCX, TXT)
- Extract **skills, education, experience** with NLP
- Match** resumes to job descriptions
- Rank and classify with **TF-IDF, Naive Bayes, SVM**
- Generate a **top-N shortlist** automatically

DATASET

- 2,400

synthetic resumes
- 12

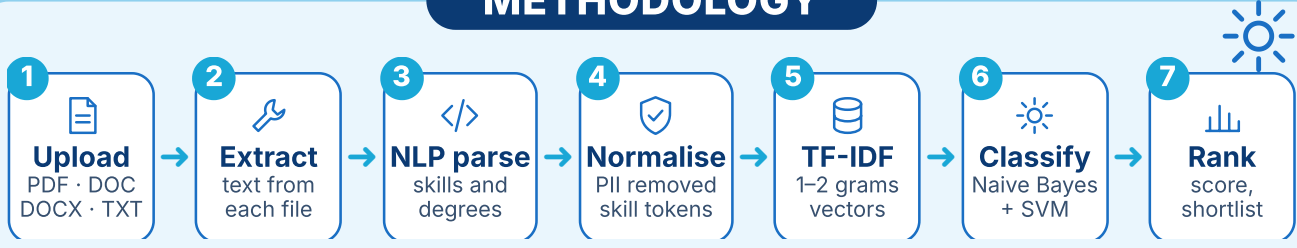
job roles · 200 each
- 11,688

TF-IDF features
- 1,920 train / 480 test** (stratified 80/20) + 5-fold CV
 - Role-catalog generator with noise: hybrid and sparse resumes

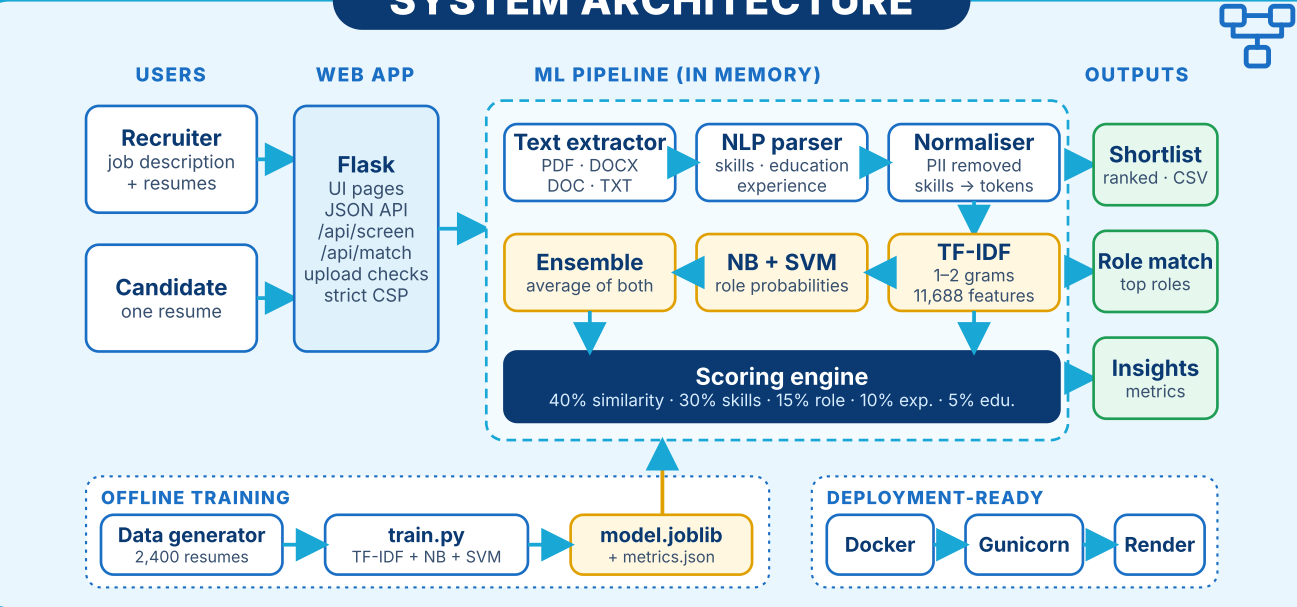
CONCLUSION

ResumelQ is a working ML web application that turns unstructured resumes into ranked, explainable shortlists and role recommendations. It screened 14 sample resumes in about 0.19 s and reached 99.8% hold-out accuracy on synthetic data; validation on real resumes is the key step before production use.

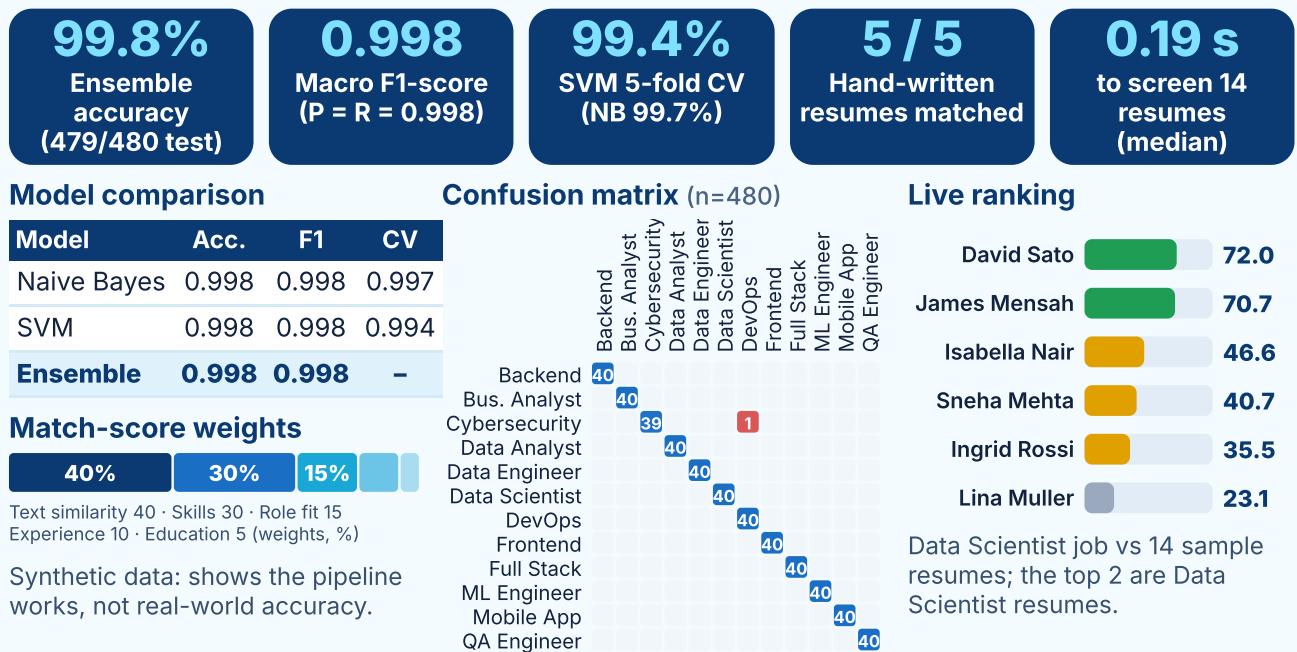
METHODOLOGY



SYSTEM ARCHITECTURE



RESULTS & PERFORMANCE



TECHNOLOGIES

Py

Python

Fl

Flask

sk

scikit-learn

Pd

Pandas

Np

NumPy

Pf

pypdf

Dx

python-docx

Jb

joblib

Js

HTML·CSS·JS

Gu

Gunicorn

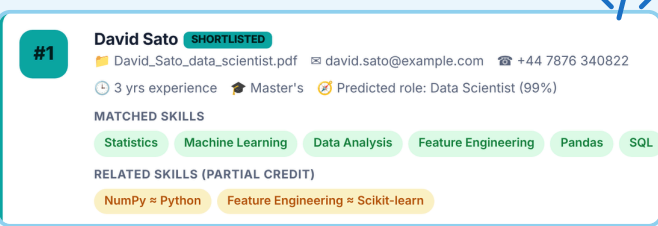
Dk

Docker

Rn

Render

IMPLEMENTATION



- Flask app: 4 pages + JSON API, strict security headers
- Rule-based NLP with a **164-skill** taxonomy; names and contacts removed before scoring
- Model trained at build time; Docker + Render blueprint; **34 tests**

DISCUSSION

- NB, SVM and the ensemble perform alike (≈ 0.998); the single error labels a **Cybersecurity** resume as **DevOps**, a neighbouring role.
- Data is synthetic, so scores are **not** a real-world accuracy estimate.
- All 5 hand-written resumes, unseen by the generator, matched the right role. Limits: rule-based extraction, no OCR.

FUTURE ENHANCEMENTS

- Train and validate on real labelled resumes
- OCR for scanned PDFs; trained NER for extraction
- Fairness audit; broader roles and skill taxonomy

REFERENCES

- Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR* 12, 2011.
- Salton & Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.* 24(5), 1988.
- McCallum & Nigam, "A comparison of event models for Naive Bayes text classification," *AAAI Workshop*, 1998.
- Joachims, "Text categorization with support vector machines," *ECML*, 1998.
- Platt, "Probabilistic outputs for support vector machines," *Adv. in Large Margin Classifiers*, 1999.
- Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008.

PROJECT TEAM

- Sadhana Shanmugam — 2104251040837
- Dhanya Sri Mohandass — 2104251040189

Faculty Guide (Mentor): Ms. Swathi L, Assistant Professor